R3.07
SQL dans un langage de programmation

Chapitre 4
Mapping objet-relationnel avec JPA



Selena Lamari et Chouki Tibermacine

BUT 2 Informatique — IUT de Vannes — Université Bretagne Sud

Plan du cours

1. Introduction à JPA

2. Relations entre objets persistants

Inconvénients de JDBC

Obligation d'écrire :

- des requêtes SQL dans le dialecte du DB vendor (Postgres, par exemple), alors qu'on est censé être indépendant lorsqu'on utilise une API standard
- des requêtes SQL répétitives, à faible valeur ajoutée, pour les opérations CRUD de base

ORM : *Object-Relational Mapping*

- Le mapping objet / relationnel vise en premier lieu à s'affranchir des deux contraintes précédentes
- Technique consistant à associer de façon déclarative, un modèle d'objets Java (ou exprimé dans un autre langage), à une structure de tables SQL, déjà existante ou non
- Ces déclarations sont habituellement faites par des annotations Java sur des classes et leurs attributs (`@Entity`, `@Table`, `@Column`, ...) – dans le passé, on utilisait XML

ORM : *Object-Relational Mapping* – suite

- Une fois cette association modèle objet-tables déclarée, l'API JPA est capable de :
  1. vérifier si la structure de tables existe bien en base de données, et si elle n'existe pas, elle est capable de la créer
  2. d'écrire l'ensemble des requêtes SQL nécessaires à la manipulation complète en base des objets Java instance des classes annotées (les persister – `INSERT`, les rechercher – `SELECT`, ...)
- La façon dont les classes Java sont annotées est indépendante de la base de données utilisée
- Possible donc d'associer des classes annotées à (à peu près) n'importe quelle BdD, sans modification du code Java

Différentes implémentations de JPA

- Hibernate : projet open source, le plus populaire, maintenu par JBoss/RedHat
  - Ce projet existait avant JPA et est à l'origine de la création de l'API, suite à son succès (à Hibernate)
- TopLink (EclipseLink) : implémentation de référence, à l'origine c'était un projet propriétaire d'Oracle, cédé plus tard à la fondation Eclipse
- OpenJPA : projet Apache

Composants de base d'une application JPA

- Des classes d'objets dits "persistants" (entités) : classes Java classiques annotées
- Un fichier de description persistence.xml, qui contient certaines informations dont l'implémentation JPA a besoin pour fonctionner
- Les dépendances à l'API JPA, à l'implem choisie (Hibernate, par ex) et au Driver JDBC : voir pom.xml fourni
- Une base de données, dans laquelle JPA va créer des tables et enregistrer des données

Pour la suite, vous devez avoir un serveur de BdD en marche (voir cours JDBC)

Environnement de l'application

- Créer un projet Maven sur IntelliJ IDEA
- Récupérer le pom.xml fourni dans la page Moodle du cours
- Ouvrir la tool-window "Database" (dans la bande latérale verticale à droite) pour créer une DataSource
- Remplir les paramètres nécessaire (hostname, port, username, password et nom de la BdD)
- Tester la connexion : il faut que ça passe au vert
- Ouvrir la tool-window "Persistence" : menu `View > Tool Windows > Persistence`
- Si cette tool-window n'existe pas :
  - créer un dossier META-INF dans le dossier resources de votre projet puis y copier le fichier persistence.xml depuis Moodle
  - copier dans votre projet le fichier User.java depuis Moodle
  - ignorer la diapo suivante

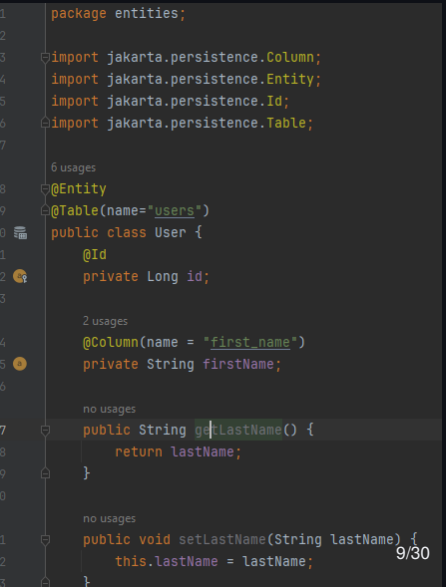
Unité de persistance

- Une unité de persistance est un ensemble de classes d'objets persistants regroupées entre elles, et associées à une même base de données
- Dans la tool-window ouverte précédemment, nous avons une unité de persistance par défaut (`default`)
- Nous allons l'utiliser pour créer des classes d'objets persistants : classes annotées Entity
- Cliquer bouton-droit sur `default > New > Entity`
- Donner un nom à la classe : `User` par exemple
- Choisir un nom de package : `entities` par exemple
- Votre IDE va créer une classe annotée `@Entity`

Classe d’objets persistants

- `@Table` : nom table
- `@Id` : associer un attribut de la classe à la clé primaire
- On peut ajouter l’annotation suivante à l’attribut annoté @Id si jamais les clés sont générées par la BdD : `@GeneratedValue(strategy = GenerationType.AUTO)`
- `@Column` : associer un attribut à une colonne dans la table

* Noter l’élément `name` des annotations



Classe d'objets persistants – contraintes

La classe annotée `@Entity` :

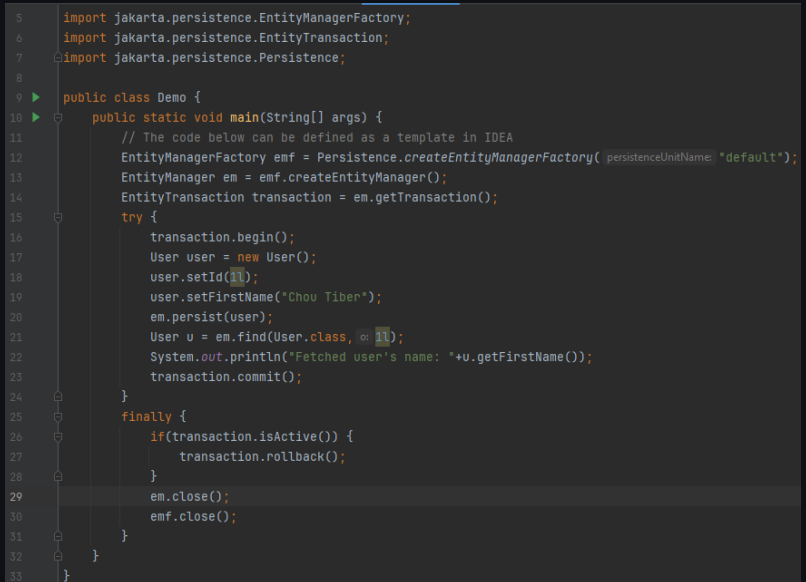
- doit posséder un constructeur vide, public ou protected. Pour rappel : ce constructeur vide existe par défaut si aucun constructeur n'existe dans la classe. Dans le cas contraire, il doit être ajouté explicitement ;
- ne doit pas être `final`, et aucune de ses méthodes ne peut être déclarée `final` ;
- ne peut pas être une interface ou une énumération ;
- peut être une classe concrète ou abstraite.

Configuration de l'unité de persistance

```xml
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd"
             version="3.0">
    <persistence-unit name="default">
        <class>entities.User</class>
        <properties>
            <property name="javax.persistence.jdbc.url" value="jdbc:postgresql://localhost:5432/jpa_demo"/>
            <property name="javax.persistence.jdbc.driver" value="org.postgresql.Driver"/>
            <property name="javax.persistence.jdbc.user" value="postgres"/>
            <property name="javax.persistence.jdbc.password" value="postgres"/>
            <property name="hibernate.show_sql" value="true"/>
            <property name="hibernate.hbm2ddl.auto" value="create"/>
        </properties>
    </persistence-unit>
</persistence>
```

- Seules les deux dernières properties (pas obligatoires) sont spécifiques à l'implémentation de JPA Hibernate
- L'avant-dernière property permet de voir les requêtes SQL exécutées
- La dernière property permet de créer des records s'ils n'existent pas dans les tables

Utiliser l'unité de persistance



```java
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.EntityTransaction;
import jakarta.persistence.Persistence;

public class Demo {
    public static void main(String[] args) {
        // The code below can be defined as a template in IDEA
        EntityManagerFactory emf = Persistence.createEntityManagerFactory( persistenceUnitName: "default");
        EntityManager em = emf.createEntityManager();
        EntityTransaction transaction = em.getTransaction();
        try {
            transaction.begin();
            User user = new User();
            user.setId(11);
            user.setFirstName("Chou Tiber");
            em.persist(user);
            User u = em.find(User.class, o: 11);
            System.out.println("Fetched user's name: "+u.getFirstName());
            transaction.commit();
        }
        finally {
            if(transaction.isActive()) {
                transaction.rollback();
            }
            em.close();
            emf.close();
        }
    }
}
```

Opérations CRUD dans JPA

- Création : `em.persist()`
- Mise à jour : pas de méthode à invoquer pour mettre à jour une valeur en base. Dès qu'un objet est persistant, et que l'on modifie un de ses champs dans le cadre d'une transaction, alors sa valeur en base sera modifiée
- Effacement : `em.remove()` en passant l'objet à supprimer en paramètre
- Recherche à partir d'une clé primaire : `em.find(...)`

Annotations des attributs

- Les attributs d'une classe `@Entity` sont automatiquement associés à des colonnes en base, dans la table de l'entité
- Ces colonnes portent les mêmes noms que ces champs
- Si ces 2 règles s'appliquent, aucune annotation n'est nécessaire
- Les types associés automatiquement à des colonnes sont :
  - Tous les types de base Java : int, long, float, etc...
  - Tous les types enveloppes (wrappers) des types de base, et le type String ;
  - Les types date java.sql.Date, java.sql.Time et java.sql.TimeStamp. Les types java.util.Date et java.util.Calendar ne sont donc pas associés automatiquement ;
  - Les types énumérés ;
  - Les types Serializable, qui sont rangés dans des blob.

Annotation `@Column`

- Annotation pouvant s'appliquer à un attribut ou à un getter
- Elle expose les attributs optionnels suivants :
  - `name` : permet d'indiquer le nom de la colonne ;
  - `length` : pour les chaînes de caractères, cet attribut permet de fixer la taille du champ SQL associé. Cette taille n'est pas vérifiée dans le code Java ;
  - `unique` : ajoute une contrainte d'unicité sur la colonne ;
  - `nullable` : empêche cette colonne de porter des valeurs nulles. Cette contrainte est utile pour la définition de clés étrangères ;
  - `insertable`, `updatable` : empêche la modification des valeurs de cette colonne, utilisée pour les clés étrangères ;
  - ...
- L'annotation `@Lob` permet de forcer l'application de cette colonne dans un blob SQL

Données de type temporel (dates, heures, ...)

- Les attributs de type `java.util.Date` et `java.util.Calendar` ne sont pas associés avec des données en base directement
- Pour préciser le type SQL, on utilise l'annotation `@Temporal`
- Cette annotation prend un unique élément :
  - `TemporalType.DATE` : enregistre les dates en jour / mois / année ;
  - `TemporalType.TIME` : enregistre les dates en heure : minute : seconde ;
  - `TemporalType.TIMESTAMP` : enregistre les deux informations de date et d'heure dans la journée
- Exemple :

```
@Column(name="date_de_naissance")
@Temporal(TemporalType.DATE)
private java.util.Date dateNaissance ;
```

Données de types énumération

- Les énumérations Java peuvent être associées à des colonnes de deux façons :
  1. Soit l'entity manager crée une colonne numérique, et stocke dedans un entier, qui correspond au numéro d'ordre de la valeur que porte le champ (efficace mais peu flexible),
  2. Soit l'entity manager crée une colonne de type chaîne de caractères, et stocke le nom de la valeur énumérée.
- On spécifie ce point avec l'annotation `@Enumerated`, qui peut prendre deux valeurs en attribut : `EnumType.ORDINAL` ou `EnumType.STRING`
- Exemple

```
@Column(name="civility")
@Enumerated(EnumType.STRING)
private Civility civilite ;
```

Opérations sur les entités

Toutes les opérations sur les entities passent par l'entity manager :

- `persist` : rendre une entité persistante, c'est à dire l'écrire en base sur le prochain commit de la transaction dans laquelle on appelle persist ;
- `remove` : rendre une entité non persistante, c'est à dire l'effacer de la base sur le prochain commit de la transaction dans laquelle on appelle remove ;
- `refresh` : synchroniser l'état d'une entité avec son état en base. Si les champs d'une entité ont été modifiés dans la transaction courante, ces modifications seront donc effacées par cette opération. Si l'entité a été modifiée en base, alors ces modifications seront prises en compte lors d'un refresh ;

Opérations sur les entités – suite

- `detach` : détacher l'entité de l'entity manager qui la gère. Cette entité persistante ne sera donc pas prise en compte lors du prochain commit de la transaction dans laquelle on appelle detach ;
- `merge` : attache l'entité à l'entity manager courant. On utilise cette opération pour associer une entité à un autre entity manager que celui qui a été utilisé pour la créer ou la lire.

Plan du cours

1. Introduction à JPA

2. Relations entre objets persistants

Mise en relation des entités

- On peut définir quatre types de relations entre entités JPA :
  1. relation 1 :1 : annotée par `@OneToOne` ;
  2. relation n :1 : annotée par `@ManyToOne` ;
  3. relation 1 :p : annotée par `@OneToMany` ;
  4. relation n :p : annotée par `@ManyToMany`.
- Les relations peuvent être unidirectionnelles ou bidirectionnelles
- Dans les relations bidirectionnelles, l'une des entités doit être maître et l'autre esclave
- Dans le cas des relations 1 :1 et n :p, on peut choisir le côté maître comme on le souhaite
- Dans les relations 1 :p et n :1, l'entité du côté 1 est l'entité esclave

Relations 1 :1

- Relation entre Patient et CarteVitale
- Cas unidirectionnel :
  - l'entité Patient comporte un attribut de type CarteVitale annoté `@OneToOne`
  - dans la BdD, la table Patient contient une colonne `num_secu` qui est une clé étrangère
- Cas bidirectionnel : l'entité esclave (CarteVitale) doit préciser un champ retour par une annotation `@OneToOne` et un attribut `mappedBy`, qui doit référencer le champ qui porte la relation côté maître (Patient)

Relations 1 :p

- Relation entre Patient et Consultation
- Une relation 1 :p est caractérisée par un champ de type Collection dans la classe maître
- La classe esclave ne porte pas de relation retour. Cette relation peut être spécifiée soit par l'annotation `@OneToMany` ou `@ManyToMany`

```
@OneToMany
private Collection<Consultation> consultations ;
```

- Elle peut être unidirectionnelle ou bidirectionnelle. Dans ce second cas, le côté maître est obligatoirement le côté qui tient la relation monovaluée

Relations 1 :p – suite

- JPA crée une table de jointure entre les deux tables (entités)
- Cette table de jointure porte une clé étrangère vers la clé primaire de la première table, et une clé étrangère vers la clé primaire de la deuxième table
- Cas bidirectionnel : on ajoute une relation p :1 dans la classe destination de la relation
- Dans ce cas : l'élément `mappedBy` est défini pour l'annotation `@OneToMany`, mais pas pour l'annotation `@ManyToOne`

```
// Dans la classe Patient
@OneToMany(mappedBy="patient")
private Collection<Consultation> consultations ;
// Dans la classe Consultation
@ManyToOne
private Patient patient;
```

Autres relations

- La relation bidirectionnelle p :1 est identique à celle (bidirectionnelle) 1 :p
- Cas d'une relation unidirectionnelle : annotation `@ManyToOne` sur un attribut de l'entité source de la relation (patient, par exemple)
- JPA crée une colonne de jointure dans la table Consultation, et une clé étrangère qui référence la clé primaire de la table Patient
- Pour les relations n :p (annotation `@ManyToMany`) JPA crée en base une table de jointure avec des clés étrangères
- C'est la présence de l'attribut `mappedBy` qui crée le caractère bidirectionnel de la relation

Comportement `cascade`

- Comportement cascade : spécifier ce qui se passe pour une entité en relation d'une entité père, lorsque cette entité père subit une des opérations `detach`, `persist`, `remove`, ...
- Comportement cascade précisé par l'élément `cascade` des annotations : `@OneToOne`, `@OneToMany` et `@ManyToMany`
- Valeur de cet élément à choisir dans l'énumération `CascadeType`
- Valeurs possibles : `DETACH`, `MERGE`, `PERSIST`, `REMOVE`, `REFRESH`, et `ALL` (toutes les valeurs précédentes)

```
@OneToOne(cascade={CascadeType.PERSIST, CascadeType.REMOVE})
private CarteVitale carte;
```

Chargement des entités et leurs relations depuis la BdD

- Est-ce que les relations doivent être chargées en même temps que les entités ?
  - Est-ce qu'on laisse faire un seul `SELECT` avec jointure ou bien des `SELECT` distincts ?
- Tout dépend de l'applicatif
- JPA nous permet d'indiquer la façon de charger avec l'élément `fetch` des annotations `@OneToOne`, ... :
  1. `FetchType.LAZY` : la relation doit être chargée à la demande ;
  2. `FetchType.EAGER` : la relation doit être chargée en même temps que l'entité qui la porte.

```
@OneToMany(fetch=FetchType.EAGER)  // la relation est chargee par defaut
private Collection<Consultation> consultations ;
```

Optimisations possibles

- Annotation `@Embeddable` au lieu de `@Entity` (et `@Embedded` au lieu de `@Column`) pour ne pas multipler les tables de jointure
  - Cas d'une `Adresse` d'un `User`, par exemple
  - On peut utiliser des *Records* Java 16+ au lieu des classes
- Annotations pour gérer les relations multivaluées particulières :
  - `@OrderBy` ou `@OrderColumn` pour les java.util.List
  - `@MapKeyColumn` pour les java.util.Map
- Gestion de l'héritage entre classes dans le modèle relationnel (stratégies `SINGLE_TABLE`, `JOINED` et `TABLE_PER_CLASS` – voir cours BdD avancées)
- Langage de req. indépendant des BdD : JPQL (*Java Persistence Query Language*) pour écrire des méthodes spécifiques

Utiliser JPA dans IntelliJ IDEA

- Lire le tuto : `https://www.youtube.com/watch?v=QJddHc41xrM`
- Le code adapté à Postgres est fourni sur le dépôt Moodle

Questions ?



Chouki.Tibermacine@univ-ubs.fr